

UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

INTELLIGENZA ARTIFICIALE

Corso di Laurea Magistrale in Ingegneria Informatica

Elaborato

PPO e GRPO

Mattia Fornari - Mat. 731764
Gabriele Festa - Mat. 734870

Anno Accademico 2024/2025

Indice

1 Introduzione **3**

2 Reinforcement Learning **4**

2.1 Fondamentali di Reinforcerment Learning 4

2.1.1 Azione a 4

2.1.2 Stato s 5

2.1.3 Traiettoria τ 5

2.1.4 Policy π 5

2.1.5 Reward r_t 5

2.1.6 Return G_t 5

2.1.7 Markov Decision Process 6

2.1.8 Value Function $V^{\pi}(s)$ 7

2.1.9 Action Value Function $Q^{\pi}(s, a)$ 8

2.1.10 Advantage Function $A^{\pi}(s, a)$ 8

2.1.11 Expectation of Reward Optimization 9

2.1.12 Finite Horizon Reward $G(\pi_\theta)$ 9

2.1.13 On-Policy 10

2.1.14 Actor-Critic 10

2.1.15 Reference Model π_{ref} 12

2.2 Metodi Policy Gradient 12

2.3 TRPO 14

3 Proximal Policy Optimization **16**

3.1 PPO-Clipped Surrogate Objective 17

3.2 PPO-Adaptive KL Penalty Coefficient 19

3.3 Algoritmo PPO 20

4 Esperimenti e Risultati **23**
4.1 Confronto degli Obiettivi Surrogati 23
4.2 Confronto con Altri Algoritmi nel Dominio Continuo 24
4.3 Dimostrazione Pratica: Umanoide in 3D 25
4.4 Confronto su Atari 26

5 Da PPO a GRPO **27**
5.1 Group Relative Policy Optimization (GRPO) 28

6 GRPO **32**
6.1 Group Relative Policy Optimization 32
6.2 Supervisione del Risultato e del Processo con GRPO 34
6.2.1 Outcome Supervision RL con GRPO 34
6.2.2 Process Supervision RL con GRPO 35
6.3 RL Iterativo con GRPO 35
6.4 Training e Valutazione di DeepSeekMath-RL 36
6.5 Lezioni Apprese nel Pre-Training 36
6.6 Il Paradigma Unificato 37
6.6.1 Analisi dei Metodi di Training 37
6.6.2 Osservazione sulla fonte dei dati 39
6.6.3 Osservazione sul Coefficiente del Gradiente 39
6.7 Efficacia e Prospettive del Reinforcement Learning 41

7 Conclusioni **42**

Capitolo 1

Introduzione

L'obiettivo di questa relazione consiste nel condurre un'analisi di due algoritmi di Reinforcement Learning: Proximal Policy Optimization (PPO) [3] e Group Relative Policy Optimization (GRPO) [6].

In primo luogo, si analizza l'algoritmo PPO concentrandosi sui dettagli implementativi del metodo, tra cui l'architettura actor-critic, le varianti PPO-Clip e PPO-Penalty, il calcolo del vantaggio e si riportano brevemente i risultati sperimentali ottenuti in [3]. Successivamente, la relazione esamina il metodo GRPO introdotto in [6], variante di PPO che migliora le capacità di ragionamento matematico ottimizzando al contempo l'utilizzo di memoria rispetto a PPO. L'analisi dimostra come GRPO, pur mantenendo lo stesso obiettivo di PPO, introduca una modifica strutturale fondamentale per risolvere i vincoli di memoria di cui sopra: l'eliminazione del modello critic. In particolare, si approfondisce il meccanismo con cui GRPO determina la baseline, optando per un calcolo basato sulla media empirica delle ricompense del gruppo, eliminando di conseguenza il processo di training di un modello separato per il valore.

Infine, nel capitolo Conclusioni si presenta un confronto dei due algoritmi con riferimento anche all'applicazione del Reinforcement Learning in ambito Large Language Models (LLM) molti grandi evidenziando come GRPO può rendere più scalabile l'allenamento del modello.

Capitolo 2

Reinforcement Learning

Il Reinforcement Learning (RL, apprendimento per rinforzo) è un'area dell'Intelligenza Artificiale (AI) che si concentra sull'addestramento di agenti che possono prendere decisioni interagendo con il loro ambiente per massimizzare una ricompensa cumulativa [4]. A differenza dell'apprendimento supervisionato, in cui un agente apprende da esempi etichettati, o dell'apprendimento non supervisionato, che si basa sul rilevamento di modelli o schemi nei dati, l'apprendimento per rinforzo si occupa di un agente autonomo che deve prendere decisioni intuitive e di conseguenza imparare dalle proprie azioni. L'idea chiave è di imparare cosa fare, ovvero come mappare le situazioni in azioni, in modo da massimizzare un segnale di ricompensa numerico nel tempo attraverso l'esplorazione per tentativi ed errori.

In questa sezione viene fornita una breve panoramica dei concetti fondamentali del Reinforcement Learning che costituiscono la base teorica e che saranno utili per il proseguimento della relazione.

2.1 Fondamentali di Reinforcerment Learning

Le seguenti definizioni sono tratte da [8, 4].

2.1.1 Azione a

Una decisione o un'azione compiuta da un agente in un determinato ambiente, spesso rappresentata come $a \in A$, dove A è l'insieme delle azioni possibili.

2.1.2 Stato s

La configurazione o situazione attuale dell'ambiente, solitamente indicata come $s \in S$, dove S è lo spazio degli stati.

2.1.3 Traiettoria τ

Una traiettoria τ (episodio o rollout) è una sequenza di stati, azioni e ricompense osservati da un agente: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, ..., s_T, a_T, r_T)$.

2.1.4 Policy π

Nell'apprendimento per rinforzo, una policy è una strategia o una regola che l'agente segue per decidere quale azione $a$intraprendere in un dato stato s: $\pi(a|s)$. Nel Reinforcement Learning in ambito Natural Language Processing, i modelli di linguaggio LLM possono essere considerati come "policy model". Può essere deterministica o stocastica. In Deep Reinforcement Learning, la policy è rappresentata da una rete neurale.

2.1.5 Reward r_t

Il reward $r_t \in \mathbb{R}$ è un segnale di feedback fornito dall'ambiente all'agente dopo aver eseguito un'azione in un dato passo temporale t che quantifica numericamente quanto l'azione sia stata "buona" o "cattiva" rispetto all'obiettivo dell'agente. Il reward è immediato e riflette la bontà dell'azione appena compiuta e viene usato dall'agente per aggiornare la propria policy.

$$r_t = R(s_t, a_t, s_{t+1})$$

Il segnale di ricompensa definisce quindi quali sono gli eventi positivi e negativi per l'agente. L'obiettivo dell'agente è massimizzare una ricompensa cumulativa su una traiettoria nel lungo periodo. In generale, i reward possono essere funzioni stocastiche dello stato dell'ambiente e delle azioni intraprese.

2.1.6 Return G_t

L'obiettivo dell'agente è massimizzare la ricompensa cumulativa che riceve nel lungo periodo. In generale, si cerca di massimizzare il rendimento (return) atteso G_t, che è

una funzione specifica della sequenza di ricompense. Indica quanta "ricompensa totale attesa" produce una determinata sequenza di azioni.

Un tipo di return è il rendimento non scontato a orizzonte finito, definito come la somma dei reward ottenuti in un intervallo fisso di passi:

$$G_t = \sum_{t=0}^{T} r_t$$

dove T è il passo finale. Un'altra forma di return è il return scontato a orizzonte infinito: ovvero la somma di tutte le ricompense che l'agente otterrà nel futuro, ciascuna moltiplicata per un fattore di sconto $\gamma \in [0, 1)$, che decresce esponenzialmente quanto più lontana è la ricompensa nel tempo:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

2.1.7 Markov Decision Process

Un Markov Decision Process (MDP) è un framework per la modellizzazione di processi decisionali. L'apprendimento per rinforzo utilizza i Markov Decsion Process per modellare l'interazione tra un agente di apprendimento e il suo ambiente per raggiungere un obiettivo attraverso stati (configurazione o situazione attuale dell'ambiente), azioni e ricompense come riportato in figura 2.1. In particolare, se gli spazi degli stati e delle azioni sono finiti, allora il problema è chiamato MDP finito.

Formalmente, un Markov Decision Process è un modello di stato probabilistico completamente osservabile. La formulazione più comune degli MDP è un Discounted-Reward Markov Decision Process (è un MDP finito). Un discount-reward MDP è una tupla $(S, s_0, A, P, R, \gamma)$ contenente:

- uno spazio degli stati S: è un insieme finito di stati $s \in S$;
- uno stato iniziale $s_0 \in S$;

- A uno spazio finito delle azioni $a \in A$: azioni $A(s) \subseteq A$ $s \in S$ che l'agente può eseguire in ogni stato;
- P è il modello dinamico/di transizione per ciascuna azione, che specifica

$$P(s_{t+1} = s' | s_t = s, a_t = a) \tag{2.1}$$

  : probabilità di transizione $P_a(s' \mid s)$ per $s \in S$ e $a \in A(s)$;
- R è la funzione di reward:

$$R(s_t = s, a_t = a) = \mathbb{E}[r_t \mid s_t = s, a_t = a] \tag{2.2}$$

  : $r(s, a, s')$ è un valore scalare (reward) che può essere positivo, negativo o nullo associato alla transizione dallo stato s allo s' usando l'azione a;
- Discount factor γ: uno scalare $\gamma \in [0, 1]$ cioè $0 \leq \gamma \leq; 1$: riduce in modo esponenziale il peso delle ricompense future nel return, bilanciando l'immediatezza con il guadagno a lungo termine e garantendo la convergenza per somme a orizzonte infinito. A volte lo sconto non viene utilizzato, il che equivale a $\gamma = 1$.

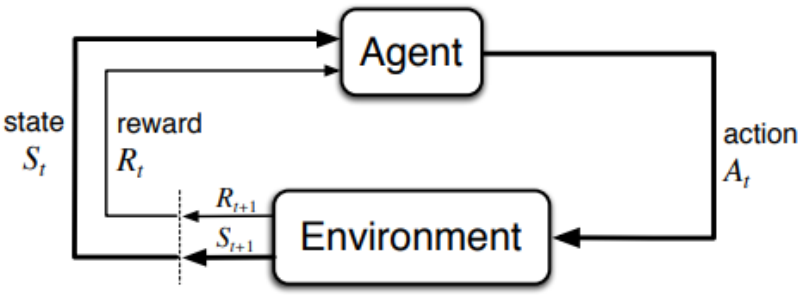


Figura 2.1: L'interazione agente-ambiente nell'apprendimento per rinforzo.

2.1.8 Value Function $V^\pi(s)$

La state-value function sotto una policy π è la funzione che stima il valore atteso del ritorno scontato quando si parte dallo stato s e si segue la policy π in tutti i passi successivi. Formalmente:

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \;\middle|\; s_0 = s\right],$$

ove s_0 è lo stato iniziale, $\gamma \in (0, 1)$ è il fattore di sconto e il valore atteso è calcolato sulle traiettorie generate da π.

2.1.9 Action Value Function $Q^{\pi}(s, a)$

La action-value function sotto una policy π è la funzione che stima il valore atteso del ritorno scontato quando si prende un'azione a nello stato s e, a partire da quel punto, si segue la policy π. Formalmente:

$$Q^{\pi}(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \, r_t \;\middle|\; s_0 = s, \; a_0 = a\right],$$

dove $\gamma \in (0, 1)$ è il fattore di sconto, e il valore atteso è calcolato sulle traiettorie generate da π dopo aver eseguito l'azione a nello stato s.

Per le funzioni V e Q esistono anche le versioni ottime V^* e Q^* (se si agisce seguendo una policy ottima) presenti nelle equazioni di Bellman, il cui approfondimento esula dagli scopi di questa relazione.

2.1.10 Advantage Function $A^{\pi}(s, a)$

Il vantaggio (advantage function) sotto una policy π misura quanto l'azione a in uno stato s sia migliore o peggiore rispetto al comportamento medio dell'agente in quello stato. La funzione non esprime un valore assoluto, bensì un valore relativo: indica quanto, in termini di ritorno futuro atteso, si guadagna o si perde scegliendo l'azione a rispetto alla media delle azioni che la policy π sceglierebbe in s. È definito come:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

dove $Q^{\pi}(s, a)$ è il valore atteso se si sceglie l'azione a nello stato s e si segue π, e $V^{\pi}(s)$ è il valore medio dello stato s sotto π.

In altre parole, $A^\pi(s,a)$ misura quanto l'azione a sia migliore o peggiore rispetto al rendimento atteso $\mathbb{E}_\pi[G_t \mid s_t = s]$ dello stato s (secondo la policy π), cioè indica quanto si discosta in meglio o in peggio dal rendimento atteso nello stato s. Ovvero:

- se $A(s,a) > 0$: l'azione a è meglio dell'azione media nello stato s e quindi rappresenta un'azione "vantaggiosa";
- se $A(s,a) < 0$: l'azione a è peggio dell'azione media per lo stato s e quindi rappresenta un'azione svantaggiosa;
- se $A(s,a) = 0$: l'azione a non è né peggio né meglio dell'azione media nello stato s e quindi è equivalente a alla scelta media della policy.

Negli algoritmi actor-critic e policy gradient, il vantaggio permette di ridurre la varianza delle stime del gradiente: invece di usare solo $Q(s,a)$, sottrarre $V(s)$ rimuove la parte costante comune a tutte le azioni nello stato s, mettendo in evidenza solo il contributo relativo dell'azione a.

portando a degli aggiornamenti della policy più stabili. La funzione advantage può dipendere da una specifica policy $A^\pi(s,a)$.

2.1.11 Expectation of Reward Optimization

L'obiettivo principale del reinforcement learning consiste nel massimizzare la ricompensa cumulativa attesa:

$$\max_\theta \ \mathbb{E}_{s\sim\rho_{\pi_\theta},\, a\sim\pi_\theta}\left[\sum_{t=0}^{\infty}\gamma^t\, r_t\right],$$

dove ρ_{π_θ} è la distribuzione degli stati indotta dalla policy π_θ e $\gamma \in [0,1)$ è il fattore di sconto.

2.1.12 Finite Horizon Reward $G(\pi_\theta)$

Il valore atteso del return scontato a orizzonte finito per la policy π_θ, parametrizzata da θ, è definito come:

$$G(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right] \qquad (5)$$

ove $\tau \sim \pi_\theta$ indica le traiettorie campionate seguendo la policy π_θ (appresenta una sequenza di stati, azioni e reward raccolti seguendo π_θ)e T è l'ultimo passo temporale (orizzonte finito).

2.1.13 On-Policy

In Reinforcement Learning, on-policy indica che l'algoritmo utilizza per l'apprendimento gli stessi dati generati dalla policy che sta cercando di ottimizzare. In altre parole, la policy utilizzata per esplorare l'ambiente e raccogliere traiettorie coincide con la policy che viene aggiornata: i dati (stati, azioni e reward) provengono direttamente dalla policy corrente. Nel contesto dei Large Language Model (LLM) e dell'addestramento tramite RLHF, on-policy significa che le risposte su cui viene calcolato il reward, e da cui deriva l'aggiornamento dei parametri della policy, sono generate dalla versione attuale del modello (la policy π_θ in uso), In questo modo la policy viene continuamente affinata in base agli output recenti, mantenendo coerenza tra il comportamento esplorato e quello appreso.

2.1.14 Actor-Critic

I metodi Actor-Critic combinano i punti di forza dei metodi value-based e policy-based. Questi algoritmi mantengono due componenti principali, spesso implementate mediante reti neurali:

- un Actor, che rappresenta la policy parametrizzata $\pi_\theta(a|s)$ responsabile della scelta delle azioni da eseguire;
- un Critic, che rappresenta una funzione di valore (tipicamente $V_\phi(s)$, oppure $Q_\phi(a, s)$) responsabile di valutare quanto "buono" è uno stato o una coppia di stato-azione.

L'idea fondamentale dell'approccio Actor-Critic è quella di apprendere contemporaneamente la policy e una funzione di valore: l'actor migliora la policy sulla base del feedback

fornito dal critic, mentre il critic approssima la funzione di valore mediante metodi di Temporal Difference (TD). L'aggiornamento dei parametri dell'actor avviene tramite policy gradient, generalmente utilizzando una stima del vantaggio $A(s, a)$, come:

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a|s) \cdot A(s, a)\right]$$

dove $A(s, a) = Q(s, a) - V(s)$. A ogni passo (o batch di passi) si aggiornano sia la policy sia la funzione di valore. Il critic viene aggiornato tramite metodi TD per ridurre l'errore nella stima della funzione di valore, tipicamente minimizzando un errore quadratico tra la stima attuale e il target TD [7].

I metodi Actor-Critic permettono di:

- ridurre la varianza delle stime del gradiente rispetto ai metodi policy-based grazie all'uso del critic come baseline;
- aggiornare sia la policy sia la funzione di valore in modo incrementale (online), senza dover aspettare la fine dell'episodio;
- gestire spazi di azione continui e discreti.

Tuttavia,questi metodi sono più complessi da implementare (due moduli da addestrare) e introducono un compromesso bias-varianza: l'approssimazione del critic può aggiungere bias alle stime del gradiente anche se riduce la varianza [7].

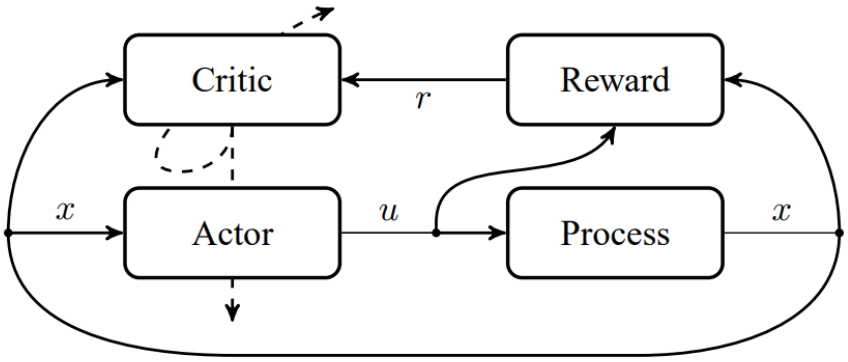


Figura 2.2: Schema di un algoritmo actor-critic. La linea tratteggiata indica che l'elemento "critic" è responsabile dell'aggiornamento dell'attore (actor) e di se stesso (Immagine presa da [1]).

2.1.15 Reference Model π_{ref}

Nel contesto del Reinforcement Learning from Human Feedback (RLHF), il "Reference Model" (o reference policy o modello di riferimento) è un insieme di parametri salvato e fisso utilizzato per regolarizzare l'ottimizzazione della policy corrente. Questa policy di riferimento è generalmente ottenuta da un modello pre-addestrato (per esempio un checkpoint ottenuto dalla fase di Supervised Fine-Tuning), e non viene aggiornata durante la fase di training mediante RL. La policy di riferimento π_{ref} serve principalmente come "ancora" di stabilità e di sicurezza nel processo di ottimizzazione durante il RLHF. I suoi output o la sua distribuzione di probabilità vengono usati come baseline per regolarizzare o limitare l'aggiornamento della policy attiva π_θ, impedendo che la nuova policy si discosti troppo (in termini di stile, sicurezza, plausibilità) dal comportamento base già accettabile (baseline). In pratica, tale regolarizzazione è spesso implementata come una penalità basata sulla divergenza di Kullback-Leibler (KL) tra la policy corrente e quella di riferimento, in modo da mantenere coerenza nello stile, nella sicurezza e nelle risposte plausibili del modello (capitolo 8 di [8]). Poiché π_{ref} è congelata, non riceve gradienti durante l'apprendimento RLHF, ma influenza l'ottimizzazione della policy attiva attraverso la regolarizzazione. Di conseguenza aiuta a prevenire che la nuova policy si discosti troppo da un comportamento base considerato accettabile.

2.2 Metodi Policy Gradient

Nei metodi Policy Gradient, l'obiettivo non è semplicemente massimizzare la ricompensa immediata, ma il reward atteso cumulativo (o return), ovvero la somma delle ricompense future (eventualmente scontate di un fattore). Il return G_t che rappresenta questa somma, a partire dal time step t, non compare esplicitamente nella formulazione standard del policy gradient.

Tuttavia, questi metodi apprendono una value-function $V(s)$ per stimare il return atteso associato a uno stato s. Queste stime sono poi utilizzate per calcolare l'advantage, che è il cuore dell'apprendimento.

Lo stimatore dell'advantage $\hat{A}_t$ è la quantità che guida l'aggiornamento della policy. Il vantaggio misura quanto un'azione a_t, effettuata nello stato s_t, rende meglio (o peggio)

rispetto al valore medio atteso per quello stato. Essenzialmente, è la differenza tra il return ottenuto da a_t e il value $V(s_t)$ dello stato.

Per aggiornare la policy, l'algoritmo di Policy Gradient cerca di risolvere il seguente problema di ottimizzazione:

$$\max_{\theta} J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[G_t] = \mathbb{E}_{\tau\sim\pi_\theta}[\sum_{t=0}^{\infty} \gamma^t r_t] \qquad (4)$$

.

utilizzando uno stimatore del gradiente all'interno di un algoritmo Stochastic Gradient Ascent sui parametri θ. Questo approccio permette di ottimizzare la policy in modo efficace, bilanciando esplorazione e sfruttamento delle azioni. Lo stimatore del gradiente più comunemente usato ha la forma:

$$\hat{g} = \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \hat{A}_t \right]$$

dove $\hat{A}_t$ è lo stimatore dell'advantage, definito come:

$$\hat{A}_t = G_t - V(s_t)$$

e la funzione obiettivo può essere espressa come:

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \right]$$

- π_θ è la policy stocastica;
- $\hat{A}_t$ è uno stimatore della funzione di vantaggio al timestep t;
- $\hat{\mathbb{E}}_t$ è la media empirica su un insieme finito di samples.

Lo stimatore $\hat{g}$ è ottenuto differenziando la funzione obiettivo $L^{PG}(\theta)$.

Una spiegazione dettagliata dei metodi policy gradient va oltre lo scopo di questo lavoro. È comunque importante sottolineare i loro principali svantaggi: questi metodi usano i dati in modo poco efficiente, perché ogni campione viene utilizzato una sola volta prima di essere scartato. Inoltre, un piccolo cambiamento nei parametri della policy può

a volte causare un peggioramento improvviso e grave delle prestazioni. Le stime del gradiente sono spesso molto incerte (rumorose), il che rende l'addestramento instabile. Esiste anche il rischio che l'apprendimento si blocchi su una soluzione non ottimale (massimo locale) e che i risultati siano molto sensibili alla scelta degli iperparametri, che quindi vanno regolati con attenzione.

2.3 TRPO

L'algoritmo TRPO (Trust Region Policy Optimization) è stato introdotto per risolvere il problema della stabilità dei metodi Policy Gradient. TRPO limita gli aggiornamenti delle policy per mantenere le nuove policy "sufficientemente vicine" a quelle vecchie (all'interno di una regione di fiducia o trust region), utilizzando la divergenza di Kullback-Leibler come metrica di distanza.

L'algoritmo prevede di massimizzare una funzione obiettivo "surrogata" con un vincolo sull'aggiornamento della policy:

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right] \quad \text{soggetto a} \quad \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]] \leq \delta$$

dove θ_{old} rappresenta i parametri della policy prima dell'aggiornamento e KL è la divergenza di Kullback-Leibler, utilizzata per evitare che la policy cambi troppo in un singolo aggiornamento, limitando la policy a rimanere in una "trust region", descritta da questa equazione, nel caso di una distribuzione di probabilità discreta:

$$\mathbb{D}_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Il rapporto $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ è chiamato *importance sampling*. È uno strumento che permette di usare i campioni raccolti dalla vecchia policy ($\pi_{\theta_{\text{old}}}$) per stimare il valore atteso della nuova policy (π_θ). Il rapporto funziona da peso di correzione: moltiplicando per esso il vantaggio di un'azione, si ottiene uno stimatore della performance della nuova policy, riutilizzando i dati passati. Questo aiuta a gestire la scarsa efficienza campionaria dei metodi policy gradient, permettendo di aggiornare la policy senza dover ricampionare continuamente.

Per controllare l'ampiezza degli aggiornamenti e garantire stabilità, TRPO limita quanto la nuova policy può differire dalla vecchia. La teoria di TRPO suggerisce di utilizzare una penalità che scoraggi grandi cambiamenti, formulando il seguente problema di ottimizzazione non vincolato:

$$\max_{\theta} \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta \, \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right]$$

dove β è un iperparametro di penalizzazione e KL rappresenta la divergenza di Kullback-Leibler.

Tuttavia, in pratica, scegliere un singolo valore di β che funzioni bene in contesti diversi è molto difficile. Per questo motivo, l'algoritmo TRPO standard non utilizza questa forma penalizzata, ma preferisce un vincolo rigido sulla massima divergenza KL permessa. Questo approccio con vincolo si è dimostrato più robusto e affidabile nella pratica.

La soluzione di questo problema di ottimizzazione con vincolo viene approssimata in modo efficiente usando l'algoritmo del gradiente coniugato, dopo aver approssimato la funzione obiettivo in modo lineare e il vincolo di divergenza KL in modo quadratico. Nonostante la sua efficacia in problemi complessi, questo approccio è computazionalmente oneroso, a causa del calcolo della matrice di informazione di Fisher e della ricerca lineare, ed è complesso da implementare.

Capitolo 3

Proximal Policy Optimization

TRPO offre aggiornamenti "sicuri" della policy (con vincolo sull divergenza KL) e garantisce miglioramenti monotoni, ma usa metodi di ottimizzazione basati sul secondo ordine (matrice di Fisher, Conjugate Gradient, line-search), che sono computazionalmente costosi e complessi da implementare [2].

PPO mira a conservare la stabilità e la sicurezza degli aggiornamenti policy-based tramite un approccio molto più semplice: una funzione obiettivo surrogata con clipping del "probability ratio", che permette l'ottimizzazione con metodi del primo ordine (gradient descent o Adam) senza calcoli pesanti, rendendo l'algoritmo scalabile, più compatibile con modelli grandi (e.g. LLM), parallelizzazione e più robusto da usare in contesti pratici. PPO è una famiglia di metodi di gradient-policy che alternano la raccolta di dati tramite interazione con l'ambiente e l'ottimizzazione di un obiettivo "surrogato" mediante Gradient Ascent Stocastico. A differenza dei policy-gradient tradizionali, PPO migliora l'efficienza campionaria, poichè consente più epoche di aggiornamento sullo stesso batch di dati, grazie a una funzione obiettivo con ratio di probabilità "clippata" che limita quanto la nuova policy può divergere da quella precedente, e di conseguenza permette di imparare meglio mediante meno interazioni con l'ambiente, fattore importante quando campionare può divenire costoso. Questi accorgimenti lo rendono più semplice da implementare, più efficiente in termini di campioni e spesso più stabile rispetto a metodi complessi come Trust Region Policy Optimization (TRPO), mantenendo comunque un buon equilibrio tra efficienza, robustezza e prestazioni.

Nel seguito della relazione, vengono presentati due varianti principali dell'algoritmo

PPO come illustrato in [3]: PPO-Clip e PPO-Penalty. Entrambe mirano a limitare il cambiamento della policy; la versione PPO-Clip è diventata uno standard grazie alla sua semplicità e alle sue ottime prestazioni.

3.1 PPO-Clipped Surrogate Objective

Sia $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ il rapporto delle probabilità, quindi $r(\theta_{\text{old}}) = 1$. TRPO massimizza la funzione obiettivo:

$$L^{\text{CPI}}(\theta) = \hat{\mathbb{E}}_t \left[r_t(\theta) \hat{A}_t \right]$$

ove il CPI fa riferimento all'algoritmo di Conservative Policy Iteration.

Tuttavia, senza vincoli questa massimizzazione porterebbe ad aggiornamenti eccessivi della policy causando un probabile crollo delle performance della stessa (l'apprendimento divergerebbe). Per penalizzare i cambiamenti che allontanano $r_t(\theta)$ da 1, la funzione obiettivo è stata modificata come segue:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

dove ϵ è un iperparametro (clip range).

La motivazione principale è duplice: il clipping rimuove l'incentivo a far uscire r_t dall'intervallo $[1-\epsilon, 1+\epsilon]$, mentre l'operatore min fornisce un limite inferiore pessimistico sull'obiettivo non clippato. Ovvero, piuttosto di imporre un vincolo per la divergenza KL, la funzione obiettivo viene modificata in modo tale da non trarre vantaggio da grandi cambiamenti nella policy. Viene aumentata la probabilità di un'azione vantaggiosa (e ridotta la probabilità di un'azione svantaggiosa), ma gli effetti del rapporto di probabilità molto superiore (o molto inferiore) a 1 sono limitati dal clipping, come illustrato più avanti. Un valore tipicamente scelto per l'iperparametro ϵ è 0.2.

L'operazione di *clip* potrebbe essere per esempio implementata, in PyTorch, mediante il metodo *torch.clamp(x, x_{min}, x_{max})* e usando successivamente *torch.min* per completare l'implementazione della loss surrogata. In questo caso, x è il rapporto di probabilità r_t che misura quanto la nuova policy differisce dalla vecchia, in termini di quanto più (meno) probabile è che prenda la stessa azione a_t nello stato s_t, x_{min} è $1-\epsilon$ e x_{max} è $1+\epsilon$. Quindi la funzione clip limita il rapporto all'intervallo $[1-\epsilon, 1+\epsilon]$, assicurando che

l'aggiornamento dei parametri della policy non sia eccessivo (e dunque che il rapporto diventi troppo grande o troppo piccolo) e penalizzando gli aggiornamenti che spingono il rapporto oltre all'intervallo sopra indicato, stabilizzando l'apprendimento.

Si noti che $L^{\text{CLIP}}(\theta) = L^{\text{CPI}}(\theta)$ al primo ordine intorno a θ_{old}, ma divergono quando θ si allontana da θ_{old}. La Figura 3.1 mostra che il fattore $1-\epsilon$ o $1+\epsilon$, a seconda che il vantaggio $\hat{A}_t$ sia positivo o negativo, stabilisce entrambi un limite superiore (upper bound) alla funzione obiettivo. Ciò può essere meglio visualizzato con quanto segue:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \begin{cases} \min\Big(r_t(\theta)\hat{A}_t,\ (1+\epsilon)\hat{A}_t\Big), & \text{se } \hat{A}_t \geq 0, \\ \min\Big(r_t(\theta)\hat{A}_t,\ (1-\epsilon)\hat{A}_t\Big), & \text{se } \hat{A}_t < 0. \end{cases}$$

L'operatore "min" serve affinché $1-\epsilon$ o $1+\epsilon$ abbiano effetto e formino il limite massimo (ceiling).

In particolare:

- se $\hat{A}_t > 0$: la funzione obiettivo L^{CLIP} è maggiore (più positiva) quando il rapporto r_t è superiore a 1, ovvero quando la nuova policy π_θ seleziona più spesso l'azione vantaggiosa. Tuttavia, esiste un limite massimo dovuto al fattore $1+\epsilon$.

- se $\hat{A}_t < 0$: la funzione obiettivo L^{CLIP} è maggiore (più negativa) quando il rapporto r_t è inferiore a 1, ovvero quando la nuova policy π_θ seleziona più spesso l'azione svantaggiosa. Tuttavia, esiste un limite massimo dovuto al fattore $1-\epsilon$.

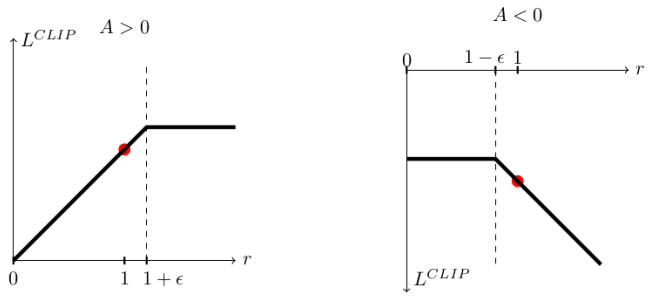


Figura 3.1: Grafici che mostrano un termine (cioè un singolo timestep) della funzione surrogata L^{CLIP} in funzione del rapporto di probabilità r, per vantaggi positivi (sinistra) e vantaggi negativi (destra). Il cerchio rosso su ciascun grafico mostra il punto di partenza per l'ottimizzazione, cioè $r = 1$.

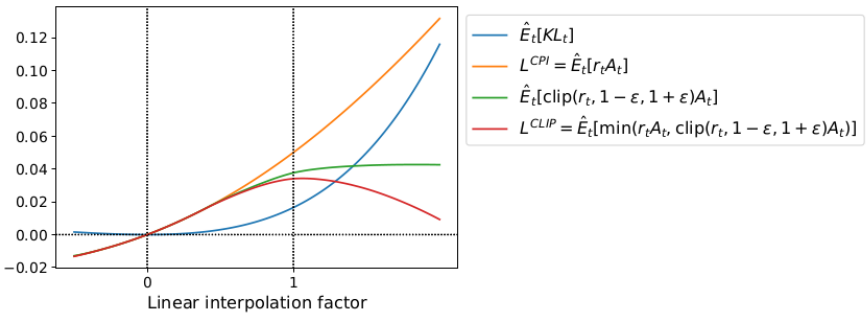


Figura 3.2: Questa figura mostra come diversi obiettivi variano interpolando lungo la direzione di aggiornamento della policy, ottenuta con l'ottimizzazione della policy prossimale su un problema di controllo continuo. Possiamo osservare che L^{CLIP} è un limite inferiore di L^{CPI}, con una penalità per aggiornamenti della policy eccessivamente grandi.

Il meccanismo di clipping in PPO di migliorare la policy consente alla policy di aumentare la probabilità delle azioni con advantage positivo (cioè le azioni che si sono dimostrate "buone") evitando cambiamenti troppo drastici in un singolo aggiornamento. Di conseguenza la policy impara a preferire le azioni con advantage positivo, ma in modo graduale e controllato Il clipping stabilizza l'apprendimento e favorisce aggiornamenti "sicuri" (non troppo grandi) invece di oscillazioni. Questo produce un compromesso tra apprendimento aggressivo (rapido miglioramento) e stabilità / robustezza: la policy evolve gradualmente, riducendo il rischio di oscillazioni o degradazioni improvvise nelle performance. Inoltre, nella pratica permette di riutilizzare lo stesso batch di dati per più epoche senza "sprecare" l'esperienza, perché i cambiamenti sono piccoli e più sicuri.

3.2 PPO-Adaptive KL Penalty Coefficient

La versione PPO-Penalty aggiunge una penalità alla funzione obiettivo proporzionale alla divergenza KL tra la nuova e la vecchia policy (e adatta questo coefficiente di penalità durante l'addestramento in modo da ottenere un valore target della divergenza KL d_{targ} a ogni aggiornamento della policy). Questo approccio è più simile a quello del TRPO, in quanto l'algoritmo cerca di mantenere bassa la divergenza KL tramite una penalità esplicita.

La funzione obiettivo con penalità KL è definita come:

$$L^{KLPEN}(\theta) = \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t - \beta \, \text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)] \right]$$

Dove:

- π_θ è la policy corrente;
- $\pi_{\theta_{\text{old}}}$ è la policy precedente;
- $\hat{A}_t$ è il vantaggio stimato;
- β è il coefficiente di penalità adattativo;
- KL è la divergenza di Kullback-Leibler.

Nella più semplice implementazione di questo algoritmo, vengono eseguiti i seguenti passaggi in ogni aggiornamento della policy [3]: utilizzando diverse epoche di minibatch SGD, si ottimizza la funzione $L^{KLPEN}(\theta)$, poi si calcola:

$$d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]]$$

L'aggiornamento di β segue questa regola:

$$\beta \leftarrow \begin{cases} \beta/2 & \text{se } d < d_{\text{target}}/1.5 \\ \beta * 2 & \text{se } d > d_{\text{target}} * 1.5 \\ \beta & \text{altrimenti} \end{cases}$$

Il nuovo valore di β viene utilizzato per l'aggiornamento successivo della policy. Con questo schema, occasionalmente si osservano aggiornamenti della policy in cui la divergenza KL è significativamente diversa da d_{targ}. Tuttavia, questi casi sono rari e l'iperparametro β si adatta velocemente. I valori 1.5 e 2.0 sono stati scelti in modo euristico; il valore iniziale di β è un altro iperparametro, ma non influisce nella fratica per via della volicità di adattamento dello stesso.

3.3 Algoritmo PPO

Nella pratica, PPO è implementato come un algoritmo on-policy actor-critic.

L'implementazione delle funzioni obiettivo PPO richiede solo modifiche minime rispetto agli approcci standard di policy gradient. Utilizzando la differenziazione automatica, è sufficiente sostituire la loss tradizionale L^{PG} con L^{CLIP} o L^{KLPEN}, per poi eseguire multiple ottimizzazioni con discesa stocastica del gradiente.

La funzione obiettivo completa combina tre componenti essenziali: la surrogate loss della policy, un termine di errore per la value function e un bonus di entropia per promuovere l'esplorazione. La formulazione risultante è:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right]$$

dove $L_t^{VF} = (V_\theta(s_t) - V_t^{\text{target}})^2$ è l'errore quadratico della value function e S rappresenta il bonus di entropia, con c_1, c_2 iperparametri per pesare rispettivamente la value functione il bonus di entropia

Il bonus di entropia può essere scritto come:

$$S[\pi_\theta](s_t) = -\sum_a \pi(a_t|s_t) log \pi(a_| s_t)$$

Poiché il PPO utilizza l'ascesa del gradiente, l'aggiunta di questo termine di entropia incoraggia l'agente a mantenere un livello di incertezza (una maggiore entropia suggerisce una maggiore incertezza) riguardo l'azione migliore, portandolo a esplorare una gamma di azioni più ampia durante l'addestramento.

Per l'implementazione pratica con segmenti di traiettoria di lunghezza fissa T, viene utilizzato uno stimatore dell'advantage che non guarda oltre l'orizzonte T versione generalizzata dello advantage estimator (GAE) è:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \cdots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$$

con

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

L'algoritmo PPO completo prevede che ad ogni iterazione N attori paralleli raccolgano T timesteps di dati, per un totale di NT campioni. Su questi dati viene costruita la surrogate loss, che viene ottimizzata per K epoche utilizzando minibatch SGD o Adam.

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Figura 3.3: Codice dell'algoritmo PPO.

Nel dettaglio, PPO alterna fasi di collezione dati (eseguendo la policy corrente nell'ambiente e raccogliendo traiettorie "stato-azione-reward") e fasi di ottimizzazione in cui calcola per ogni transizione un'advantage (usando tipicamente GAE come riportato sopra) e poi aggiorna la policy massimizzando una "surrogate objective" $L_t^{CLIP+VF+S}(\theta)$ che utilizza un rapporto di probabilità tra la nuova e vecchia policy, con il meccanismo di clipping che stabilizza laggiornamento. Durante lo stesso batch, si possono fare più epoche di update (minibatch SGD o Adam) invece di un update per campione, e contemporaneamente si aggiorna il critic attraverso una loss sul valore $V(s_t)$ stimato, migliorando le stime del vantaggio per le iterazioni successive. L'aggiornamento di actor e critic è separato: l'actor (policy model) viene aggiornato tramite gradient ascent su L_t^{CLIP}. In pratica si applica un algoritmo di ottimizzazione come SGD / Adam per massimizzare la performance attesa della policy. Il critic (value model) invece viene aggiornato minimizzando la value loss L_t^{VF} tramite gradient descent (SGD o variante). Questo schema garantisce un buon bilanciamento tra efficienza campionaria (riuso minibatch dei dati campionati), stabilità e semplicità di implementazione, rendendo PPO uno dei metodi più usati nel deep RL.

Capitolo 4

Esperimenti e Risultati

In questo capitolo vengono riportati i risultati sperimentali del metodo PPO presentati in [3].

4.1 Confronto degli Obiettivi Surrogati

Gli autori hanno condotto un confronto sistematico con diverse varianti dell'obiettivo surrogato e con diversi iperparametri. Oltre alla funzione L^{CLIP} proposta, vengono testate versioni senza clipping o penalità, e varianti con penalità KL (sia fissa che adattiva). Per poter confrontare le varianti dell'algoritmo con diversi iperparametri, hanno usato un benchmark computazionalmente poco costoso costituito da sette task di robotica simulata in OpenAI Gym con engine MuJoCo. Ogni algoritmo è stato addestrato per 1 milione di timesteps. Oltre agli iperparametri relativi al clipping e alla penalità KL che vengono ottimizzati, gli altri valori sono fissi. Per rappresentare la policy, gli autori hanno utilizzato un MLP con due layer nascosti da 64 unità, attivazioni tanh per introdurre non linearità e producendo in output la media di una distribuzione Gaussiana; non si condividono i parametri tra la policy e la value function e non viene usato un bonus di entropia. Ogni algoritmo è stato valutato su ciascun task con 3 semi casuali, calcolando la ricompensa media totale negli ultimi 100 episodi. I punteggi sono stati normalizzati in modo che la policy casuale valga 0 e il migliore risultato 1; la media su 21 run produce un singolo valore per ciascun setting. Come riportato in 4.1, senza clipping o penalità, il

punteggio risulta negativo in un ambiente (half cheetah) che è peggio della policy iniziale casuale.

algorithm	avg. normalized score
No clipping or penalty	-0.39
Clipping, $\epsilon = 0.1$	0.76
Clipping, $\epsilon = 0.2$	**0.82**
Clipping, $\epsilon = 0.3$	0.70
Adaptive KL $d_{\text{targ}} = 0.003$	0.68
Adaptive KL $d_{\text{targ}} = 0.01$	0.74
Adaptive KL $d_{\text{targ}} = 0.03$	0.71
Fixed KL, $\beta = 0.3$	0.62
Fixed KL, $\beta = 1.$	0.71
Fixed KL, $\beta = 3.$	0.72
Fixed KL, $\beta = 10.$	0.69

Figura 4.1: Risultati del benchmark di controllo continuo. Punteggi medi normalizzati (su 21 esecuzioni dell'algoritmo, su 7 ambienti) per ogni impostazione di algoritmo/iperparametro. β è stato inizializzato a 1.

4.2 Confronto con Altri Algoritmi nel Dominio Continuo

Successivamente, PPO (PPO-clip) è stato confrontato con diversi metodi all'avanguardia per problemi di controllo continuo, tra cui TRPO, CEM e versioni di A2C. Utilizzando i parametri ottimali identificati nella sezione precedente ($\epsilon = 0.2$), PPO ha dimostrato prestazioni superiori rispetto a tutti gli altri algoritmi in quasi in tutti gli ambienti di controllo valutati, confermandosi come un metodo particolarmente efficace per questo tipo di problemi.

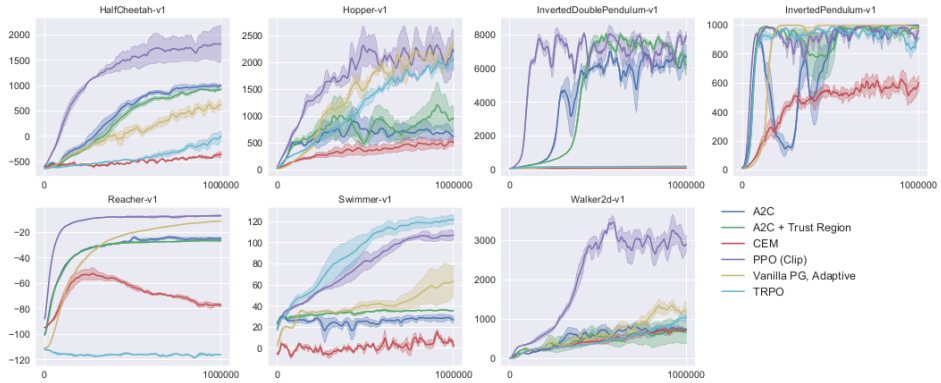


Figura 4.2: Confronto di diversi algoritmi su diversi ambienti MuJoCo, addestrati per un milione di step.

4.3 Dimostrazione Pratica: Umanoide in 3D

Per evidenziare le capacità di PPO in problemi ad alta dimensionalità in un dominio continuo, gli autori hanno presentato dei risultati su tre compiti complessi che coinvolgono un umanoide 3D. I compiti includono la locomozione di base, la navigazione verso target dinamici e una versione più difficile con ostacoli esterni. I risultati hanno dimostrato che PPO è in grado di apprendere delle policy efficaci per tutti e tre i compiti, aprendo interessanti prospettive per applicazioni robotiche complesse.

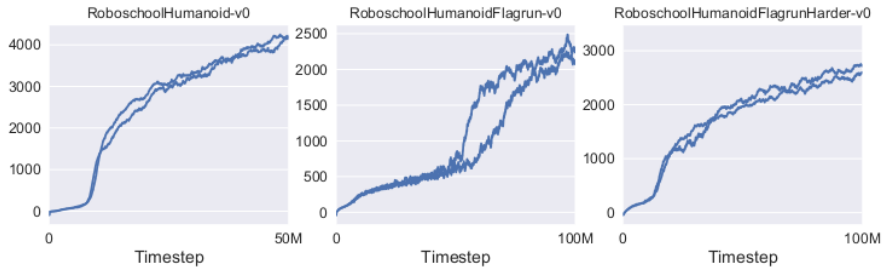


Figura 4.3: Curve di apprendimento di PPO su compiti di controllo per un umanoide 3D, utilizzando Roboschool.

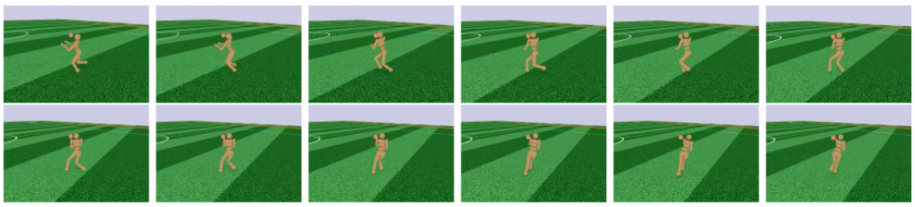


Figura 4.4: Fotogrammi fissi della policy appresa in RoboschoolHumanoidFlagrun. Nei primi sei fotogrammi, il robot corre verso un obiettivo. Successivamente, la posizione viene cambiata casualmente e il robot si gira e corre verso il nuovo obiettivo.

4.4 Confronto su Atari

L'analisi condotta dagli autori si estende anche al dominio Atari, dove PPO viene confrontato con A2C e ACER su 49 giochi diversi. I risultati mostrano che mentre PPO vince in 30 giochi considerando la ricompensa media sull'intero processo di training (indicando apprendimento più rapido), ACER prevale in 28 giochi considerando solo gli ultimi 100 episodi (ottenendo una miglior performance finale). Questo ha suggerito che PPO offre un ottimo compromesso tra velocità di apprendimento e qualità della soluzione finale.

	A2C	ACER	PPO	Tie
(1) avg. episode reward over all of training	1	18	**30**	0
(2) avg. episode reward over last 100 episodes	1	**28**	19	1

Figura 4.5: Numero di partite "vinte" da ciascun algoritmo, dove il punteggio è calcolato come media di tre prove.

Capitolo 5

Da PPO a GRPO

Nel panorama dell'addestramento dei Large Language Models per compiti di ragionamento matematico complesso, la fase di Reinforcement Learning successiva al fine-tuning supervisionato (SFT) si è rivelata cruciale per spingere ulteriormente le prestazioni. Il passaggio dallo standard Proximal Policy Optimization all'innovativo Group Relative Policy Optimization rappresenta un'evoluzione significativa, mirata a coniugare massima efficacia e sostenibilità computazionale.

L'algoritmo PPO è stato a lungo il pilastro per il potenziamento delle capacità dei LLM. L'obiettivo è di massimizare un *surrogate objective* che penalizza grandi deviazioni dalla policy precedente (*old policy*):

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}\left[\frac{1}{|o|}\sum_{t=1}^{|o|}\min\left(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t\right)\right] \tag{5.1}$$

dove: $r_t(\theta)$ è il probability ratio e A_t è il vantaggio (*advantage*).

La ricompensa per token è data da:

$$r_t = r_\theta(q, o_{\leq t}) - \beta \log \frac{\pi_\theta(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})} \tag{5.2}$$

dove r_θ è il modello di ricompensa e il termine con β è una penalità di KL rispetto a un modello di riferimento π_{ref}.

Nonostante la sua efficacia, il PPO presenta due svantaggi cruciali in contesti resource-intensive come l'addestramento di LLM:

1. Onere computazionale e di memoria: richiede un modello "critico" (value function) aggiuntivo, di dimensioni paragonabili al modello "attore" (policy), per stimare il valore degli stati. Questo virtualmente raddoppia l'uso della memoria GPU;

2. complessità nell'addestramento: addestrare una value function precisa è difficile, specialmente in un dominio come quello dei LLM dove tipicamente solo il token finale riceve un reward segnale da un modello di reward esterno. Stimare un valore accurato per ogni token intermedio può essere instabile e inefficiente.

Inoltre, un ulteriore motivo particolarmente importante per applicazioni quali il ragionamento matematico con LLM, è che la natura del reward e del processo di generazione rende poco intuitivo usare un critic token-per-token: la correttezza e la qualità di una risposta si valutano solo quando la generazione è completa, non passo dopo passo.

5.1 Group Relative Policy Optimization (GRPO)

Il GRPO, introdotto dagli autori di DeepSeekMath [6], si propone come una variante efficiente del PPO che mitiga le limitazioni descritte in precedenza con lo scopo di migliorare l'efficienza e la stabilità dell'apprendimento nei LLM, in particolare per compiti di ragionamento matematico.

L'intuizione chiave del GRPO è di eliminare il modello critic, calcolando una baseline direttamente dai reward stessi, sfruttando la natura comparativa del problema, invece di fare affidamento su una stima appresa della funzione valore.

Formalmente, per ogni prompt q, il GRPO campiona un gruppo di G risposte $\{o_1, o_2, \cdots, o_G\}$ dalla policy corrente. Un modello di reward r_φ assegna un punteggio a ciascuna risposta, producendo un vettore di reward $\mathbf{r} = \{r_1, r_2, \cdots, r_G\}$ laddove:

- Il baseline è la media dei reward del gruppo: $\text{mean}(\mathbf{r})$.
- La scala è la deviazione standard del gruppo: $\text{std}(\mathbf{r})$.
- Il reward normalizzato per la i-esima risposta è: $\overline{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$.

Questo reward normalizzato $\overline{r}_i$ viene poi utilizzato direttamente come stima dell'advantage $\hat{A}_{i,t}$ per tutti i token nella risposta o_i nella funzione obiettivo semplificata:

$$\mathcal{J}_{GRPO}(\theta) \approx \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t}\right) - \beta\mathbb{D}_{\mathrm{KL}}\left[\pi_\theta||\pi_{ref}\right]\right] \tag{5.3}$$

GRPO rispetto a PPO presenta questi vantaggi:

- Efficienza delle risorse: rimuovendo il modello (la rete neurale) critic, si riduce drasticamente il consumo di memoria e il carico computazionale, rendendo l'addestramento più accessibile;
- allineamento con i modelli di reward: l'approccio "relativo al gruppo" si sposa perfettamente con come vengono addestrati i modelli di reward, tipicamente su dataset di confronti (e.g. "risposta A è preferita a risposta B");
- regolarizzazione della divergenza KL diretta: a differenza del PPO che aggiunge una penalità KL al reward, complicandone il calcolo, il GRPO incorpora la divergenza KL direttamente nella funzione di perdita, utilizzando uno stimatore non distorto.

L'applicazione del GRPO al modello DeepSeekMath-Instruct 7B ha prodotto miglioramenti sostanziali, dimostrandone l'efficacia pratica:

- Miglioramenti In-Domain: sul benchmark MATH, l'accuratezza è salita dal 46.8% al 51.7%, un guadagno assoluto di quasi 5 punti percentuali, avvicinandosi alle prestazioni di modelli chiusi mastodontici come GPT-4;
- generalizzazione Out-of-Domain: il modello ha migliorato le sue prestazioni anche su benchmark *non* visti durante la fase di RL (e.g. CMATH), indicando che mediante GRPO non si "overfittano" i dati di addestramento RL ma che permette di potenziare le capacità di ragionamento di base del modello.

Il possibile passaggio da PPO al GRPO segna un progresso importante nell'ottimizzazione dei LLM tramite RL. Il GRPO mantiene la solidità e l'efficacia del suo predecessore mentre ne affronta le principali debolezze pratiche: l'inefficienza e la complessità. Questo lo rende uno strumento estremamente promettente per la comunità open-source, permettendo di raggiungere prestazioni all'avanguardia nel ragionamento matematico (e

potenzialmente in altri domini) con un impiego di risorse notevolmente più contenuto, democratizzando così l'accesso a tecniche avanzate di ottimizzazione per modelli linguistici di grandi dimensioni.

Model	Size	English Benchmarks		Chinese Benchmarks	
		GSM8K	MATH	MGSM-zh	CMATH
Chain-of-Thought Reasoning					
Closed-Source Model					
Gemini Ultra	-	94.4%	53.2%	-	-
GPT-4	-	92.0%	52.9%	-	86.0%
Inflection-2	-	81.4%	34.8%	-	-
GPT-3.5	-	80.8%	34.1%	-	73.8%
Gemini Pro	-	86.5%	32.6%	-	-
Grok-1	-	62.9%	23.9%	-	-
Baichuan-3	-	88.2%	49.2%	-	-
GLM-4	-	87.6%	47.9%	-	-
Open-Source Model					
InternLM2-Math	20B	82.6%	37.7%	-	-
Qwen	72B	78.9%	35.2%	-	-
Math-Shepherd-Mistral	7B	84.1%	33.0%	-	-
WizardMath-v1.1	7B	83.2%	33.0%	-	-
DeepSeek-LLM-Chat	67B	84.1%	32.6%	74.0%	80.3%
MetaMath	70B	82.3%	26.6%	66.4%	70.9%
SeaLLM-v2	7B	78.2%	27.5%	64.8%	-
ChatGLM3	6B	72.3%	25.7%	-	-
WizardMath-v1.0	70B	81.6%	22.7%	64.8%	65.4%
DeepSeekMath-Instruct	7B	82.9%	46.8%	73.2%	84.6%
DeepSeekMath-RL	7B	**88.2%**	**51.7%**	**79.6%**	**88.8%**
Tool-Integrated Reasoning					
Closed-Source Model					
GPT-4 Code Interpreter	-	97.0%	69.7%	-	-
Open-Source Model					
InternLM2-Math	20B	80.7%	54.3%	-	-
DeepSeek-LLM-Chat	67B	86.7%	51.1%	76.4%	85.4%
ToRA	34B	80.7%	50.8%	41.2%	53.4%
MAmmoTH	70B	76.9%	41.8%	-	-
DeepSeekMath-Instruct	7B	83.7%	57.4%	72.0%	84.3%
DeepSeekMath-RL	7B	**86.7%**	**58.8%**	**78.4%**	**87.6%**

Figura 5.1: Prestazioni dei modelli Open-Source e Closed-Source con ragionamento Chain-of-Thought e ragionamento con integrazione di strumenti su benchmark inglesi e cinesi. I punteggi in grigio indicano voti a maggioranza con 32 candidati; gli altri sono punteggi Top1. DeepSeekMath-RL 7B supera tutti i modelli open-source da 7B a 70B, così come la maggior parte dei modelli closed-source. Sebbene DeepSeekMath-RL 7B sia stato addestrato ulteriormente solo su dati di instruction tuning in formato chain-of-thought di GSM8K e MATH, migliora rispetto a DeepSeekMath-Instruct 7B su tutti i benchmark.

Capitolo 6

GRPO

6.1 Group Relative Policy Optimization

È stato dimostrato che l'apprendimento per rinforzo (RL) può migliorare le capacità di ragionamento matematico dei LLM dopo la fase di fine-tuning supervisionato (SFT). In questa sezione, viene introdotto l'algoritmo GRPO, che risulta sia efficiente che efficace.

Come spiegato nel Capitolo 5 dell'articolo [6], l'uso di PPO richiede una funzione valore V di dimensioni comparabili al modello della policy, con conseguente elevato onere computazionale e di memoria.

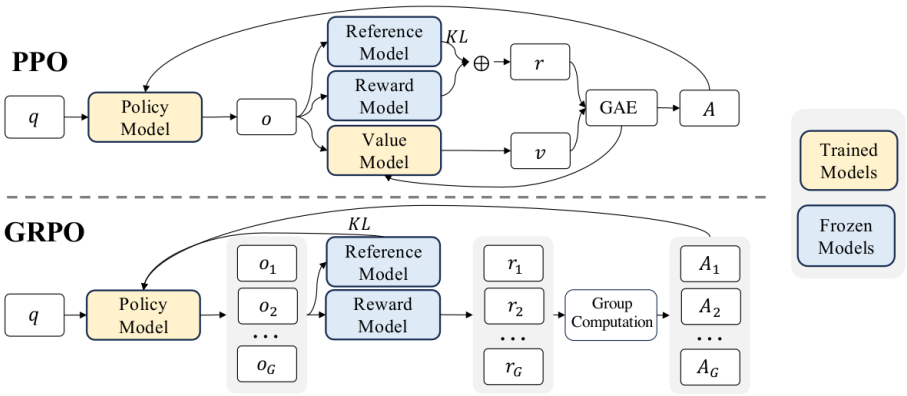


Figura 6.1: Confronto della struttura dell'algoritmo PPO e GRPO. GRPO rinuncia al modello critic (di valore), stimando invece la baseline dai group scores e riducendo significativamente le risorse di training e l'utilizzo della memoria.

Per affrontare questi problemi, viene proposto il Group Relative Policy Optimization (GRPO), che elimina la necessità di approssimare una funzione valore. Invece di utilizzare un critic, GRPO campiona un gruppo di output $\{o_1, o_2, \cdots, o_G\}$ per la stessa query q dalla vecchia policy $\pi_{\theta_{old}}$ e utilizza la ricompensa media del gruppo come baseline. L'obiettivo da massimizzare diventa:

$$\begin{aligned}\mathcal{J}_{GRPO}(\theta) = \; & \mathbb{E}\left[q \sim P(Q),\, \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)\right] \\ & \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\left\{\min\left[\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})}\hat{A}_{i,t},\right.\right. \\ & \left.\left.\text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q,o_{i,<t})},\; 1-\epsilon,\; 1+\epsilon\right)\hat{A}_{i,t}\right] - \beta\,\mathbb{D}_{KL}\left[\pi_\theta \| \pi_{ref}\right]\right\}\end{aligned} \tag{6.1}$$

dove $\hat{A}_{i,t}$ è il vantaggio (advantage) calcolato basandosi sulle ricompense relative all'interno del gruppo e α, β sono degli iperparametri. Invece di aggiungere una penalità basata sulla divergenza KL alla ricompensa, GRPO regolarizza direttamente aggiungendo la divergenza KL tra la policy addestrata e quella di riferimento alla funzione di perdita. La divergenza KL è stimata in modo non distorto e garantendo che sia positiva con:

$$\mathbb{D}_{KL}\left[\pi_\theta || \pi_{ref}\right] = \frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - 1 \tag{6.2}$$

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_φ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

policy model $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
for iteration = 1, ..., I **do**
  reference model $\pi_{ref} \leftarrow \pi_\theta$
  for step = 1, ..., M **do**
    Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
    Update the old policy model $\pi_{\theta_{old}} \leftarrow \pi_\theta$
    Sample G outputs $\{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(\cdot \mid q)$ for each question $q \in \mathcal{D}_b$
    Compute rewards $\{r_i\}_{i=1}^{G}$ for each sampled output o_i by running r_φ
    Compute $\hat{A}_{i,t}$ for the t-th token of o_i through group relative advantage estimation.
    for GRPO iteration = 1, ..., μ **do**
      Update the policy model π_θ by maximizing the GRPO objective (Equation 21)
  Update r_φ through continuous training using a replay mechanism.

Output π_θ

Figura 6.2: Algoritmo di GRPO.

Questo approccio si allinea bene con la natura comparativa dei modelli di ricompensa, in cui le decisioni vengono prese valutando le differenze tra le ricompense ottenute da diverse azioni o stati.

6.2 Supervisione del Risultato e del Processo con GRPO

L'algoritmo GRPO può essere declinato in due varianti principali basate sulla granularità della supervisione (Outcome o Process), ma entrambe condividono la medesima procedura iniziale di campionamento e valutazione del gruppo.

Per ogni query q, viene campionato un gruppo di output $\{o_1, o_2, \cdots, o_G\}$ dalla vecchia policy $\pi_{\theta_{old}}$.

6.2.1 Outcome Supervision RL con GRPO

Un modello di ricompensa assegna un punteggio agli output, producendo G ricompense $r = \{r_1, r_2, \cdots, r_G\}$. Queste ricompense vengono normalizzate sottraendo la media del gruppo e dividendo per la deviazione standard del gruppo. La supervisione del risultato fornisce la ricompensa normalizzata alla fine di ogni output o_i e imposta i vantaggi $\hat{A}_{i,t}$ di tutti i token nell'output come la ricompensa normalizzata, cioè $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$, ottimizzando quindi la policy massimizzando l'obiettivo definito nell'equazione (6.1).

6.2.2 Process Supervision RL con GRPO

La supervisione del solo risultato finale potrebbe non essere sufficiente per compiti matematici complessi. Secondo Wang et al. [5], è possibile esplorare anche la supervisione del processo, che fornisce una ricompensa alla fine di ogni passo di ragionamento. Data la query q e G output campionati $\{o_1, o_2, \cdots, o_G\}$, un modello di ricompensa di processo assegna un punteggio a ogni passo degli output, producendo ricompense corrispondenti:

$$R = \{\{r_1^{index(1)}, \cdots, r_1^{index(K_1)}\}, \cdots, \{r_G^{index(1)}, \cdots, r_G^{index(K_G)}\}\}, \tag{6.3}$$

dove $index(j)$ è l'indice del token finale del j-esimo passo e K_i è il numero totale di passi nell'i-esimo output. Normalizziamo queste ricompense con media e deviazione standard:

$$\tilde{r}_i^{index(j)} = \frac{r_i^{index(j)} - \text{mean}(R)}{\text{std}(R)} \tag{6.4}$$

Il vantaggio di ogni token viene calcolato come la somma delle ricompense normalizzate dai passi successivi:

$$\hat{A}_{i,t} = \sum_{index(j) \geq t} \tilde{r}_i^{index(j)} \tag{6.5}$$

e la policy viene ottimizzata massimizzando l'obiettivo dell'equazione (3).

6.3 RL Iterativo con GRPO

Man mano che il training mediante reinforcement learning procede, il vecchio modello di ricompensa potrebbe non essere più sufficiente per supervisionare la policy corrente. Per questo motivo, gli autori hanno proposto un approccio di reinforcement learning iterativo con GRPO come mostrato nell'algoritmo in figura 6.2. In particolare, vengono generati nuovi dataset di training per il modello di ricompensa basandosi sui risultati di campionamento della policy e vengono addestrati continuamente il vecchio modello di ricompensa utilizzando un meccanismo di replay che incorpora il 10% dei dati storici. Viene impostato quindi il modello di riferimento come modello di policy e si continua ad addestrare il modello di policy con il nuovo modello di ricompensa.

6.4 Training e Valutazione di DeepSeekMath-RL

I dati per il training mediante Reinforcement Learning sono delle query in formato Chain-of-Thought (CoT) relative a GSM8K e MATH dai dati SFT, per un totale di circa 144K query. Il modello di reward è stato addestrato inizialmente basandosi su DeepSeekMath-Base 7B con learning rate di 2e-5. Per GRPO, il learning rate del modello di policy è stato impostato a 1e-6, il coefficiente KL a 0.04 e sono stati campionati 64 output per query di lunghezza massima 1024 e batch size di 1024.

Il modello DeepSeekMath-RL 7B raggiunge accuratezze elevate (88.2% su GSM8K e 51.7% su MATH), superando tutti gli altri modelli open-source (7B-70B) e la maggior parte dei modelli closed-source. Nonostante sia stato addestrato solo con dati limitati di instruction tuning, le sue performance superiori rispetto alla versione Instruct confermano l'efficacia dell'apprendimento per rinforzo [6].

6.5 Lezioni Apprese nel Pre-Training

Gli autori riportano che l'analisi dei dati di pre-training ha portato a due scoperte fondamentali riguardanti l'efficacia del codice e dei paper accademici nel migliorare le capacità matematiche:

- Il codice migliora il ragionamento matematico: L'addestramento su codice (Code Training) non potenzia solo le abilità di programmazione, ma migliora significativamente il ragionamento matematico sia con l'uso di tool che senza. In particolare, una strategia di addestramento a due fasi (prima codice, poi matematica) si è rivelata la più efficace per massimizzare le prestazioni;

- Inefficacia dei paper arXiv: contrariamente alle pratiche comuni, l'addestramento su corpus basati su paper di arXiv non ha mostrato miglioramenti significativi nel ragionamento matematico. I modelli addestrati su arXiv non hanno registrato benefici tangibili (e talvolta hanno mostrato peggioramenti) sui benchmark standard come GSM8K e MATH rispetto all'uso di dati web filtrati.

6.6 Il Paradigma Unificato

Nell'articolo di Shao et al. [6] viene proposto un paradigma unificato per analizzare diversi metodi di training come SFT, RFT, DPO, PPO e GRPO. In generale, il gradiente rispetto al parametro θ di qualsiasi metodo di training può essere espresso come:

$$\nabla_\theta \mathcal{J}_\mathcal{A}(\theta) = \mathbb{E}\underbrace{[(q,o) \sim \mathcal{D}]}_{\text{Data Source}} \left(\frac{1}{|o|} \sum_{t=1}^{|o|} \underbrace{GC_\mathcal{A}(q,o,t,\pi_{rf})}_{\text{Gradient Coefficient}} \nabla_\theta \log \pi_\theta(o_t|q,o_{<t}) \right) \tag{6.6}$$

Il framework identifica tre componenti fondamentali. La prima componente $\mathcal{D}$ rappresenta i dati di training utilizzati. La seconda componente è la funzione di ricompensa (di reward) π_{rf}, che rappresenta la fonte del segnale di ricompensa per il training. La terza componente è l'algoritmo $\mathcal{A}$, che processa i dati di training e il segnale di ricompensa per produrre il coefficiente del gradiente GC.

6.6.1 Analisi dei Metodi di Training

Il Supervised Fine-tuning (SFT) effettua il fine-tuning del modello preaddestrato su dati selezionati da esseri umani. Il Rejection Sampling Fine-tuning (RFT) esegue un ulteriore fine-tuning sul modello SFT utilizzando output filtrati campionati dal modello SFT stesso. La Direct Preference Optimization (DPO) raffina il modello SFT mediante fine-tuning su output aumentati campionati dal modello SFT, utilizzando una loss DPO pairwise. L'Online Rejection Sampling Fine-tuning (Online RFT) inizializza il modello di policy con il modello SFT e lo raffina mediante fine-tuning con output aumentati campionati dal modello di policy in tempo reale. Infine, PPO/GRPO inizializza il modello di policy con il modello SFT e lo rinforza con output campionati dal modello di policy in tempo reale.

Methods	Data Source	Reward Function	Gradient Coefficient
SFT	$q, o \sim P_{sft}(Q, O)$	-	1
RFT	$q \sim P_{sft}(Q), o \sim \pi_{sft}(O\|q)$	Rule	Equation 10
DPO	$q \sim P_{sft}(Q), o^+, o^- \sim \pi_{sft}(O\|q)$	Rule	Equation 14
Online RFT	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Rule	Equation 10
PPO	$q \sim P_{sft}(Q), o \sim \pi_\theta(O\|q)$	Model	Equation 18
GRPO	$q \sim P_{sft}(Q), \{o_i\}_{i=1}^{G} \sim \pi_\theta(O\|q)$	Model	Equation 21

Figura 6.3: La fonte dei dati e il coefficiente del gradiente di diversi metodi. P_{SFT} denota la distribuzione dei dati dei dataset di supervised fine-tuning. $\pi_{\theta_{SFT}}$ e π_θ denotano rispettivamente il modello con supervised fine-tuning e il modello di policy in tempo reale durante il processo di training online.

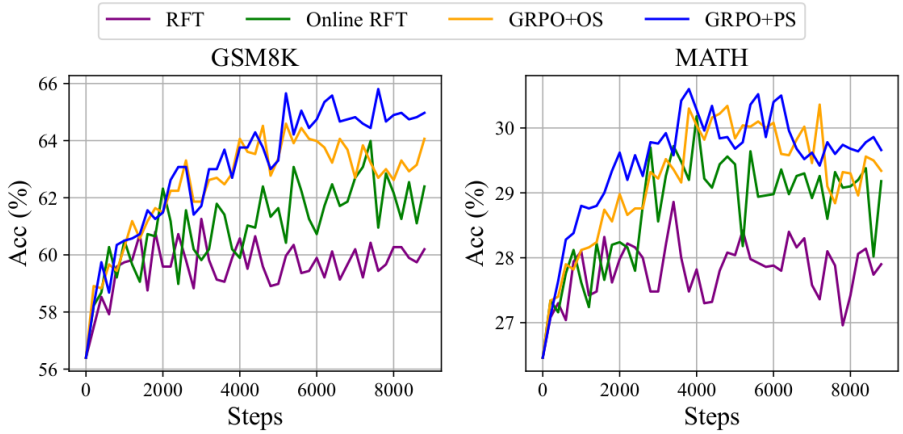


Figura 6.4: Prestazioni del modello DeepSeekMath-Instruct 1.3B, ulteriormente addestrato utilizzando vari metodi, su due benchmark.

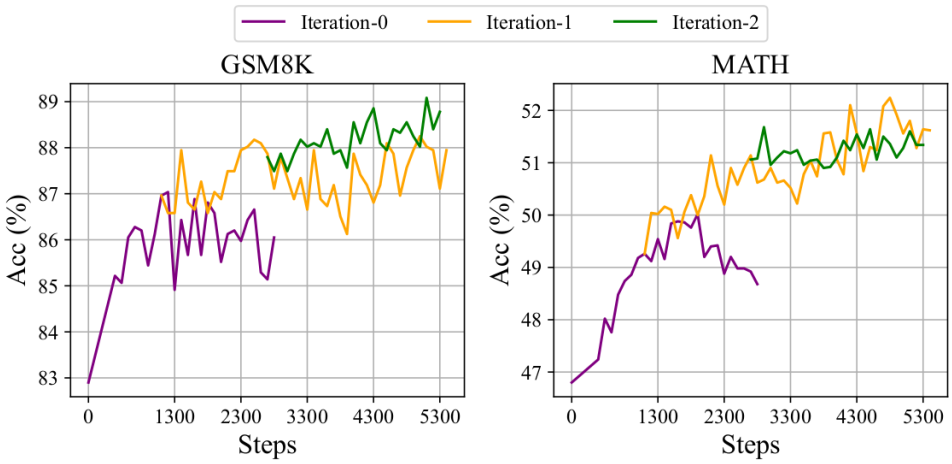


Figura 6.5: Prestazioni dell'apprendimento per rinforzo iterativo con DeepSeekMath-Instruct 7B su due benchmark.

6.6.2 Osservazione sulla fonte dei dati

La fonte dei dati viene classificata in due categorie principali. Il campionamento offline indica che i dati di training provengono dai risultati di campionamento del modello SFT iniziale, come avviene in RFT e DPO. Il campionamento online indica invece che i dati di training provengono dai risultati di esplorazione del modello di policy in tempo reale, come implementato in Online RFT e GRPO. L'Online RFT supera significativamente l'RFT standard, guadagnando un vantaggio decisivo soprattutto nelle fasi avanzate dell'addestramento. Questo miglioramento è dovuto al campionamento in tempo reale: mentre inizialmente i dati campionati (del modello actor e del modello SFT) sono simili, nelle fasi successive l'Online RFT sfrutta le divergenze emergenti tra il modello corrente (actor) e quello iniziale (SFT), offrendo dati di addestramento più vantaggiosi.

6.6.3 Osservazione sul Coefficiente del Gradiente

L'algoritmo processa il segnale di ricompensa nel coefficiente del gradiente per aggiornare i parametri del modello. Nelle sperimentazioni, le funzioni di ricompensa sono state classificate in due categorie: "Regola" e "Modello". La categoria Regola

si riferisce al giudizio della qualità di una risposta basandosi esclusivamente sulla correttezza della risposta finale, mentre la categoria Modello denota il training di un modello di ricompensa specifico per valutare ciascuna risposta. È importante notare che i dati di training per il modello di ricompensa sono comunque basati su giudizi derivati da regole.

$$GC_{RFT}(q, o, t) = \mathbb{I}(o) = \begin{cases} 1 & \text{se la risposta di } o \text{ è corretta} \\ 0 & \text{se la risposta di } o \text{ è errata} \end{cases} \tag{6.7}$$

$$GC_{GRPO}(q, o, t, \pi_{\theta_{rm}}) = \hat{A}_{i,t} + \beta \left(\frac{\pi_{ref}(o_{i,t}|o_{i,<t})}{\pi_{\theta}(o_{i,t}|o_{i,<t})} - 1 \right) \tag{6.8}$$

Le equazioni 6.7 e 6.8 evidenziano una differenza fondamentale tra GRPO e Online RFT: GRPO regola in modo univoco il suo coefficiente di gradiente in base al valore di ricompensa fornito dal modello di ricompensa. Questa caratteristica consente un rinforzo differenziato e una penalizzazione delle risposte in accordo con le loro diverse qualità intrinseche. Al contrario, Online RFT è privo di questa funzionalità; non penalizza le risposte incorrette e rinforza uniformemente tutte le risposte corrette con lo stesso livello di intensità, indipendentemente dalla loro effettiva qualità.

Come dimostrato nella Figura 6.5, GRPO supera Online RFT, sottolineando in tal modo l'efficienza derivante dalla modifica dei coefficienti di gradiente positivi e negativi in modo dinamico. In aggiunta, GRPO con Process Supervision (PS) mostra prestazioni superiori rispetto a GRPO con Outcome Supervision (OS), indicando i tangibili benefici dell'utilizzo di coefficienti di gradiente granulari e consapevoli del singolo passo di ragionamento. Inoltre, è stata esplorata la strategia dell'Apprendimento per Rinforzo iterativo, nella quale sono stati condotti due round di iterazione. Come mostrato nella Figura 6, si osserva che l'RL iterativo migliora significativamente le prestazioni del modello, in modo particolare durante la prima iterazione.

6.7 Efficacia e Prospettive del Reinforcement Learning

Gli autori riportano che gli esperimenti condotti evidenziano che il Reinforcement Learning migliora la metrica Maj@K (la consistenza delle risposte) ma non la Pass@K. Questo indica che il RL non accresce le capacità fondamentali intrinseche del modello, ma ne migliora la robustezza, rendendo la distribuzione degli output più solida e favorendo l'emersione di risposte corrette già presenti nelle ipotesi migliori (TopK).

Basandosi sul paradigma unificato (fonte dei dati, algoritmo e funzione di Ricompensa), gli autori hanno identificato le seguenti aree critiche per migliorare l'apprendimento oltre la semplice stabilizzazione della distribuzione (Maj@K):

- Fonte dei dati (Data Source): superare l'attuale campionamento "naive" sui dati di istruzione. È necessario adottare strategie di decodifica avanzate (es. tree-search) ed esplorare prompt fuori distribuzione (OOD) per arricchire i dati. Inoltre, l'implementazione di tecniche di inferenza efficiente è cruciale per massimizzare l'efficienza dell'esplorazione della policy;
- algoritmi: poiché i segnali di reward non sono sempre affidabili (persino i dataset annotati contengono errori) , gli algoritmi non devono fidarsi ciecamente del reward model. In futuro potrebbero essere utilizzati algoritmi robusti al rumore e approcci di allineamento "Weak-to-Strong", che permetterebbero di apprendere efficacemente anche da segnali imperfetti;
- funzione di ricompensa (Reward Function): nell'articolo si delineano tre direzioni fondamentali per i reward models: migliorare la generalizzazione per gestire query fuori distribuzione e output complessi, integrare la stima dell'incertezza per collegare modelli di ricompensa deboli ad algoritmi di apprendimento forti e investire su Process Reward Models di alta qualità per fornire segnali di addestramento granulari su ogni passo del ragionamento.

Capitolo 7

Conclusioni

In conclusione, il confronto tra Proximal Policy Optimization (PPO) e Group Relative Policy Optimization (GRPO) evidenzia un trade-off determinante per l'applicazione del Reinforcement Learning ai Large Language Models (LLM), specialmente in domini complessi come il ragionamento matematico. Sebbene entrambi gli algoritmi condividano l'obiettivo finale di massimizzare la ricompensa media attesa, divergono radicalmente nella gestione della baseline. PPO adotta l'architettura Actor-Critic, delegando la stima della baseline $V(s)$ a una rete neurale dedicata (il critic). Tale approccio, pur garantendo stabilità, oneroso dispendio di risorse: la necessità di mantenere in memoria un modello aggiuntivo, raddoppia quasi l'occupazione di VRAM creando un collo di bottiglia strutturale che compromette la scalabilità nell'addestramento degli LLM.Al contrario, GRPO rappresenta un adattamento algoritmico di PPO progettato per ottimizzare l'utilizzo delle risorse di memoria. Eliminando strutturalmente il modello critic (e dunque il calcolo della stima di $V(s)$), l'algoritmo ridefinisce il calcolo della baseline come segue: per ogni singolo prompt (query) di input, viene generato un gruppo di output distinti e la baseline viene determinata istantaneamente come la media delle ricompense ottenute da questo specifico set di risposte. GRPO accetta un maggiore onere in fase di inferenza (richiedendo campionamenti multipli per lo stesso prompt) in cambio di un risparmio significativo di memoria. Tale efficienza si traduce in una capacità di scalabilità superiore, rendendo fattibile l'addestramento di LLM con dimensioni e batch size che risulterebbero proibitivi per la configurazione standard

di PPO. GRPO emerge dunque come una soluzione ingegneristica superiore a PPO per scalare l'ottimizzazione delle policy con LLM di grandi dimensioni.

L'approccio per il calcolo della baselina proposto da GRPO (basato sulla media empirica delle ricompense di un gruppo di output) si dimostra particolarmente efficace nel ragionamento matematico, un ambito dominato da reward sparsi e binari (corretto/errato). La struttura della loss sfrutta questa caratteristica: confrontando tra loro diverse risposte allo stesso prompt, l'algoritmo genera un segnale di vantaggio relativo molto più netto rispetto alla stima di un valore atteso astratto. Insieme alla penalizzazione KL, che mantiene la policy ancorata al modello di riferimento evitando il reward hacking, questo meccanismo permette al modello di discriminare efficacemente tra percorsi di ragionamento validi ed errati, garantendo una convergenza stabile anche in scenari a esito binario.

Nonostante i notevoli progressi, restano alcune limitazioni da affrontare: DeepSeekMath mostra capacità relativamente più deboli in geometria e dimostrazione di teoremi rispetto ai modelli closed-source, e la sua performance few-shot non migliora significativamente rispetto allo zero-shot. Queste osservazioni suggeriscono la necessità di migliorare ulteriormente la pipeline di selezione dei dati per costruire corpus pre-addestrati di qualità più alta, delineando così una direzione promettente per futuri avanzamenti nell'ottimizzazione di modelli per compiti di reasoning complessi.

Bibliografia

[1] Ivo Grondman et al. «A Survey of Actor-Critic Reinforcement Learning: Standard and Natural Policy Gradients». In: *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* (2012). DOI: 10.1109/TSMCC.2012.2218595.

[2] John Schulman et al. «Trust Region Policy Optimization». In: *Proceedings of the 32nd International Conference on Machine Learning*. A cura di Francis Bach e David Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, lug. 2015, pp. 1889–1897. URL: https://proceedings.mlr.press/v37/schulman15.html.

[3] John Schulman et al. «Proximal Policy Optimization Algorithms». In: *arXiv* (lug. 2017). DOI: 10.48550/arXiv.1707.06347. eprint: 1707.06347.

[4] Richard S. Sutton e Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[5] Peiyi Wang et al. «Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations.» In: *CoRR* abs/2312.08935 (2023). URL: http://dblp.uni-trier.de/db/journals/corr/corr2312.html#abs-2312-08935.

[6] Zhihong Shao et al. «DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models». In: *arXiv* (feb. 2024). DOI: 10.48550/arXiv.2402.03300. eprint: 2402.03300.

[7] DDS Notes. *Actor-Critic Methods*. Accessed: 2025-12-24. 2025. URL: https://dds.com.np/docs/machine%20learning/reinforcement%20learning/actor-critic.

[8] Nathan Lambert. *Reinforcement Learning from Human Feedback*. Online, 2025. URL: https://rlhfbook.com.